

TicketMesh

Universal Configurable Ticketing & Reservation Platform — Scope Document

Executive Summary

TicketMesh is a universal, configurable, multi-provider ticketing and reservation SaaS that can be bought as software, self-hosted, or run as a full managed service with hosting. It uses a marketplace (Uber/PickMe-style) model where shops and agents connect via app, upload and sell their services, and customers also self-serve. The product is global and white-label themeable (country, currency, language, logo, colors — all by configuration, no code), delivered across a consumer Web channel and a mobile-responsive PWA app channel, plus an admin portal. It ships with 24/7 support, fraud detection, auto issue detection and auto-fix, full SDLC documentation, test and production API docs, and deploys to AWS, Kubernetes/EKS, on-premises and Docker with horizontal autoscaling.

TicketMesh — Master Requirement Register (Single Source of Truth)

- > This file is the authoritative, complete record of every requirement for the
- > TicketMesh universal ticketing platform. It is the single source of truth and is
- > kept in sync on every change. The final .docx / .pdf scope documents are
- > rendered from this content by tools/generate_docs.py. It also drives the reusable
- > "platform prompt" (docs/PLATFORM_PROMPT_TEMPLATE.md) used to build other products.

0. Product identity & positioning

- **Product name:** TicketMesh — Universal Configurable Ticketing & Reservation Platform.
- Brand tagline: **"All your tickets, one platform."**
- Marketed as a global, production-standard SaaS that can be: (a) bought as software, (b) self-hosted on the buyer's own server, or (c) run as a full managed service (hosting included) by the vendor.
- Target deploy targets: AWS, Kubernetes (EKS), on-premises, Docker, single server — with horizontal autoscaling and easy deployment everywhere.
- Prefer free / standard production SaaS-level tooling and self-hostable components (Docker, PostgreSQL/H2, Redis, Kafka, DuckDB, Prometheus, Grafana, local/offline AI).
- Sold as a business to any buyer: SaaS marketplace model, theme/white-label marketplace, managed-service offering, and on-prem licensing (WordPress-marketplace-like).

1. Core scope — the full 53-part master blueprint

The blueprint is not compressed here. Every one of the 53 parts is retained and tracked.

Honest per-part status lives in docs/blueprint-coverage.md (BUILT / PARTIAL / DOC / PLANNED).

The authoritative blueprint document is bundled alongside:

docs/spec/TicketMesh_Full_53_Part_Development_Blueprint.pdf (and .docx).

Part 1 — Product Vision & Scope

- Universal, configurable, multi-provider ticketing & reservation platform.
- Buy-as-software / self-host / managed-service with hosting.
- Sellable production standard; portfolio + visa evidence; startup/SaaS path.

Part 2 — Global Benchmark & Product Positioning

- Benchmarked against global ticketing/reservation leaders and OTA aggregators (Omio-style aggregation model, Ticketmaster-style inventory/event model).
- Differentiator: universal multi-domain aggregation + white-label SaaS + AI/ML + no-code, backed by domain-agnostic configuration, provider adapters and AI-native workflows.

Part 3 — Universal Ticketing Model

- Core entities: Tenant, TenantBranding, Provider, ProviderProduct (ticket product), TicketType, Inventory, Offer, Reservation, Booking, Ticket, Policy,

Price, Fare, Addition/Fee, Tax, Promotion, Review, LoyaltyAccount.

Part 4 — Multi-Domain Support

- Domains: bus, train, movie, events, sports, flight, ferry, attractions, activities — each represented by configurable TicketType + capability + seat-layout semantics.
- Domain specifics (per blueprint):
 - **Bus:** routes, boarding/drop-off points, luggage rules, seat layout.
 - **Train:** stations, coaches, classes, berths, quota management.
 - **Movie:** cinema, screen, show/session, seat category, food add-ons.
 - **Events / sports:** venue, sections, rows, seats, general admission, VIP, season tickets.

Part 5 — Multi-Provider Aggregation

- Parallel search across connected providers; normalize, dedupe, compare, rank, sort.
- Universal UniversalOffer normalized view regardless of source provider.

Part 6 — Provider Adapter Framework

- Adapter contract covering: search, availability, seat-map, price, hold, release, book, confirm, get-booking, cancel, refund, modify, ticket-retrieval, validation.
- ProviderOfferClient (RestClient) + WireMock stubs for offline dev.

Part 7 — Provider Onboarding

- Provider registration + capability declaration + credentials + status (active/suspended).
- Onboarding fields (per blueprint): provider code, country, currency, timezone, API endpoint, auth mode, credentials reference, capabilities, rate limits, timeout, retries, circuit breaker, webhook/polling and reconciliation settings.

Part 8 — Capability System

- Providers declare supported capabilities; client routes only to capable providers.
- Capabilities: SEARCH, AVAILABILITY, SEAT_MAP, HOLD, BOOK, CANCEL, REFUND, MODIFY, TICKET_RETRIEVAL, VALIDATION, DYNAMIC_PRICING, REAL_TIME_INVENTORY, WEBHOOKS.
- Unsupported operations are handled explicitly (client routes only to capable providers).

Part 9 — Configuration-Driven Product Engine

- No-code/low-code: ticket products, ticket types, fields, seat layouts, policies, fees, taxes, promotions, notifications, validations — all configuration-driven via admin, no code changes required.

Part 10 — Universal Search

- Unified search across all products/domains/providers with filters, sort, range, day.

Part 11 — AI Conversational Search

- Natural-language booking intent: e.g. "bus from Colombo to Kandy tomorrow 6 pm".
- Offline-first intent parse + provider search + ranked offer response.

Part 12 — Multi-Ticket / Multi-Service Trip

- A Trip bundles multiple Booking and Journey items across domains into one unified itinerary (train + bus + movie + event combinations), GET /api/trips.

Part 13 — Smart Trip Planner

- Cross-domain journey planning (bus+train+event) with ordering and optimization hints.
- AI plans transport and activities based on time, budget, preferences and constraints, producing an itinerary with bookable options.

Part 14 — Inventory Management

- Inventory state machine: AVAILABLE -> HELD -> CONFIRMED; HELD -> EXPIRED -> AVAILABLE.
- Seat maps per product; per-tenant isolation; optimistic concurrency.
- Inventory kinds (per blueprint): seat inventory, capacity inventory, general admission, and provider synchronization of stock.

Part 15 — Reservation & Concurrency

- Idempotency keys, optimistic locking, hold expiry, reservation release scheduler, reconciliation — prevent double-selling at scale.

Part 16 — Booking Engine

- Universal booking flow: offer -> hold -> payment -> confirm -> issue ticket.
- Bookings across all domains through one engine; compensating actions for partial failures (timeout/failure -> release/compensate -> reconciliation).

Part 17 — Payment Abstraction

- Payment provider abstraction: cards, wallets, bank, refunds, webhooks.
- Pluggable payment gateway adapters (WireMock-stubbed in dev); async status poll/webhook; idempotency and country/currency-specific routing; sandbox/mock providers for free dev.

Part 18 — Refund & Cancellation

- Cancellation policies per product/ticket-type; refund calculation; partial refunds.
- Provider-specific rules, cancellation windows, refund status tracking and reconciliation.

Part 19 — Ticket Issuing & Validation

- Issue QR/barcode tickets deterministically (ZXing); /api/tickets/{id}/validate for gate validation; secure, tamper-evident payload.
- Ticket status, validation events, optional provider-issued ticket references and offline validation capability for suitable ticket types.

Part 20 — Pricing & Fare Engine

- Pricing modes (per blueprint): fixed, tiered, time-based, demand-based, promotional and AI-recommended pricing via PricingService.
- Base price + configurable fare components: fees, taxes, service fees, currency; price as CurrencyAmount (atomic currency + amount); multi-currency aware.

- Separate base fare, taxes, service fees, discounts and ****provider commission****.

Part 21 — Promotion Engine

- Coupons and promo rules (percentage / flat / min-purchase), scoped to tenant/domain; applied at checkout, validated, single use per booking.
- Rule types (per blueprint): early bird, weekday, student, group, loyalty, provider campaigns, coupons and targeted offers — rules kept configurable.

Part 22 — Customer & Loyalty

- Loyalty accounts per customer; earn points on bookings; levels; redeemable.
- Customer self-serve + agent-facilitated purchase both supported.
- Customer profile scope (per blueprint): profiles, saved passengers, booking history, preferences, favorites, wallet/credits, loyalty points, memberships, coupons and consent management.

Part 23 — Admin & Operations Console

- Admin portal: manage tenants, providers, agents, shops, products, monitor system, view analytics, audit, support, security ops.

Part 24 — No-Code / Low-Code Configuration

- Admin UI + API to configure themes, products, fees, policies — no code.
- Configuration versions auditable and deployable independently of application binaries.

Part 25 — Multi-Tenancy SaaS

- Tenant isolation (data scoping by tenant), per-tenant white-label branding, per-tenant configuration, per-tenant currency/language/locale, subscription status.

Part 26 — AI Assistant

- Conversational assistant (offline-first deterministic model; adapter for LLM later).

Part 27 — AI Agent Architecture

- Controlled tools, least privilege, financial confirmation, audit trail of tool calls.
- Tool set (per blueprint): search, availability, reservation, booking, cancellation, refund and analytics — each with tool schemas, explicit permission policy and audit.

Part 28 — AI Recommendation Engine

- Product/service recommendations from history + popularity (offline deterministic + ML).

Part 29 — AI Seat Recommendation

- Suggest seats from preference/layout heuristics (offline).
- Uses screen/vehicle geometry, price, seat category, availability, group size and user preferences; supports movie, bus, train and event layouts.

Part 30 — AI Price Prediction

- Predict future price moves from trend/heuristic model (offline + ML).

- Uses historical sales, seasonality, day/time, inventory, event attributes and price history; exposes confidence and never presents predictions as guarantees.

Part 31 — Dynamic Pricing Intelligence

- Configurable fare rules / surge heuristics applied at pricing time.
- Operator-side recommended pricing with guardrails: min/max price boundaries and full auditability of every price change.

Part 32 — AI Fraud & Cyber Risk

- Fraud scoring 0-100, signals, auto-block $\geq$ threshold, admin override, audit.
- Signals (per blueprint): abnormal booking velocity, bot-like activity, account take-over patterns, payment anomalies, promo abuse, suspicious device/IP behavior and ticket scalping patterns.
- Cyber-risk awareness integrated with security ops.

Part 33 — Disruption & Recovery Intelligence

- Detect disruption indicators (provider failures, delays, cancellations, inventory mismatches) and propose recovery options.
- Identify affected bookings, propose alternatives, notify customers and initiate rebooking workflows. GET /api/ops/disruption.

Part 34 — Ticket Intelligence / Data Analytics

- Trend analytics: bookings, revenue, demand, top products/domains, retention.
- Business dashboards (per blueprint): revenue, bookings, ticket sales, refunds, cancellations, conversion, AOV, provider performance, inventory utilization and customer behavior.

Part 35 — AI Data Analyst

- Natural-language questions over a read-only semantic warehouse layer (offline).
- Question -> metric mapping -> validated SQL -> warehouse -> statistical analysis -> chart/explanation/forecast; enforces row-level security and query limits.

Part 36 — Data Platform

- Event-capture (outbox/events) into analytical storage; PostgreSQL transactions; Parquet/DuckDB offline analytics; S3 for durable cloud data; optional warehouse later.

Part 37 — ML Platform

- Demand forecasting, cancellation prediction, fraud scoring, recommendation, anomaly.
- Pure-Java/Python deterministic implementations (no heavy runtime dependency) + pluggable.
- Track datasets, features, model versions, evaluation metrics and deployment status.

Part 38 — Event-Driven Architecture

- Outbox-style event emission on domain events; async consumers/scheduler.
- Event catalog (per blueprint): BookingCreated, ReservationHeld, ReservationExpired,

PaymentCompleted, TicketIssued, TicketCancelled, RefundCompleted,
InventoryChanged, ProviderFailed, FraudAlert.

Part 39 — Microservices Architecture

- Modular single-core delivered this build; microservice-ready boundaries. Documented splitting path.
- Recommended service topology (per blueprint): gateway, identity, tenant, catalog, ticket-type configuration, provider, provider-integration, search, availability, inventory, reservation, booking, pricing, payment, refund, ticket, notification, promotion, fraud, trip, analytics, recommendation and AI.

Part 40 — API Gateway & Edge

- Central entry-point pattern, auth filter, tenant resolution, rate limiting, routing contract, request correlation and API versioning. (In-service edge layer; dedicated gateway optional.)

Part 41 — Security Architecture

- OAuth2/JWT, RBAC (CUSTOMER/AGENT/ADMIN), tenant isolation, field-level redaction, audit log, rate limiting, input validation, encryption, PII minimization, secrets via env, security headers, CSRF-stateless API.
- Per blueprint: OAuth2/OIDC, RBAC/ABAC, tenant isolation, encryption, secrets management, audit trails, secure headers, rate limiting, mTLS where appropriate, PII minimization and zero-trust service access.

Part 42 — AI Security

- Prompt-injection defense, PII redaction, tool authorization, output validation, retrieval filtering, model allowlist, audit, adversarial eval.
- Per blueprint: prompt-injection defense, system prompt isolation, PII redaction, tool authorization, output validation, retrieval filtering, data-loss prevention, model allowlists, AI audit logs and adversarial evaluation.

Part 43 — DevOps / DevSecOps

- CI, security pipeline (Gitleaks/OWASP/Checkov), release automation, IaC.
- Full pipeline (per blueprint): build, test, scan, SBOM, containerization, artifact publication, GitOps deployment, DAST, progressive rollout, smoke tests and rollback — security checks are mandatory gates.

Part 44 — CI/CD Pipeline

- GitHub Actions CI (Java test + frontend build) on push/PR; security scans; release on tag.
- Stage map (per blueprint): lint/format -> compile -> unit -> ArchUnit -> dependency scan -> Gitleaks -> Checkov -> Syft SBOM -> container build -> Trivy/Grype -> Testcontainers -> Pact -> Playwright -> k6 -> publish -> Argo CD staging -> ZAP -> approval -> Argo Rollouts production -> smoke/rollback.

Part 45 — Kubernetes & GitOps

- K8s manifests + Helm chart + HPA autoscaling + optional Argo Rollouts.
- Per blueprint: Amazon EKS target, Helm packaging, Argo CD desired-state, Argo Rollouts canary/blue-green, KEDA for Kafka/event workloads, HPA for services, cautious VPA and Network Policies for east-west restrictions.

Part 46 — AWS Cloud Architecture

- Terraform: ECS Fargate (or EKS), ALB, RDS, scaling policies, CloudWatch, IAM.
- Per blueprint: Route 53 at DNS, edge/ALB/API Gateway at ingress, EKS for services, RDS PostgreSQL, DynamoDB for selected key-value cases, S3 for object/data-lake storage, Lambda for lightweight asynchronous jobs and CloudWatch for AWS-native monitoring.

Part 47 — Free / Offline Development

- Offline AI model (deterministic), H2 in-memory DB for tests, WireMock provider/payment stubs — start MVP without paid APIs.
- Per blueprint: Docker/Compose, PostgreSQL, Redis, Kafka, MinIO, DuckDB, Parquet, Prometheus, Grafana, Jaeger and local AI models; mock bus/train/movie providers and a deterministic payment simulator. LLM/provider adapters sit behind an AI Gateway.
- ****Never commit API keys**** — use environment variables and secret managers.

Part 48 — Frontend Web & Mobile Strategy

- Consumer Web (React + TypeScript PWA installable = web + app channel), Agent shop portal, Admin portal. Responsive + PWA offline shell.
- Per blueprint: React + TypeScript primary web consumer + admin portal; if a native mobile app is added later, React Native reuses the TypeScript domain/API concepts.

Part 49 — Testing & Quality

- Unit tests (real assertions), integration tests, WireMock integration tests, fraud/support/AI tests. Documented test strategy.
- Toolchain (per blueprint): JUnit, Mockito, ArchUnit, REST Assured, Testcontainers, Pact, Playwright, k6 and JMH. Test concurrency, duplicate callbacks, provider timeout, partial booking, refund failure, inventory mismatch and reconciliation.

Part 50 — Observability

- Health endpoints, actuator metrics, structured logs, request tracing id, monitoring docs + prometheus/grafana config.
- Per blueprint: OpenTelemetry tracing, Jaeger, Prometheus, Grafana, Micrometer and structured JSON logs including traceId, spanId, correlationId, tenantId, providerId and bookingId; monitor both technical and business SLIs.

Part 51 — Repository / Git Format

- Monorepo layout: src (backend modules), frontend, docs, infra, tools, CI.

- Branching (per blueprint): main/develop/feature or trunk-based protected main, pull requests, CODEOWNERS, conventional commits, semantic versioning and release tags.

Part 52 — Project Delivery Roadmap

- Phased roadmap from MVP to production SaaS (per blueprint):
P1 foundation/local platform; P2 identity/gateway/configuration; P3 provider framework/search/inventory; P4 booking/payment/tickets; P5 bus/train/movie simulators; P6 analytics; P7 RAG/AI assistant/agents; P8 ML/fraud/recommendations; P9 EKS/GitOps/AWS; P10 security, performance, chaos and production hardening.

Part 53 — MVP + Definition of Done + Portfolio Evidence

- MVP scope, Definition of Done, and portfolio/evidence artifacts.
- MVP (per blueprint): auth, configurable ticket type, three mock providers, multi-provider search, normalized offers, seat/availability, hold/expiry, booking, mock payment, QR ticket, cancellation/refund, Kafka events, React UI, admin configuration, observability, CI security scans and local AI.
- Done = code + tests + security + observability + docs + deployment + failure handling.
- Evidence artifacts: publish ADRs, diagrams, threat model, benchmarks, AI evaluation, provider failure tests and a reproducible setup.

2. Additional requirements given during the build session (all MUST be in docs)

2.1 Marketplace / agent model (Uber / PickMe style)

- **Customer / Agent / Admin** roles all exist.
- Any **shop / agent** can connect its service and sell to customers.
- A customer can also use the platform directly by themselves (self-serve).
- Agents/shops connect **via app**, upload their **ticket details and services**, publish products to sell tickets to customers.
- **Shop owner** can build/manage their shop using just their **app**; admin manages overall and can **monitor** everything.
- Shop self-service: publish product + inventory + seat info + price from the agent app.

2.2 Global / white-label / themeable (WordPress / Uber / PickMe style)

- Standard **global product** usable from any country, any currency, any language.
- **Everything configurable** like WordPress themes or Uber/PickMe: theme colors, images, **logo**, typography, and anything else the shop needs — purely via configuration (no code).
- **Multi-currency** first-class (CurrencyAmount = currency code + amount; per-tenant default currency; configurable exchange).
- **Multi-language / i18n** first-class (per-tenant default locale + region/country; translation dictionaries; client-selectable language).
- **Country / region** scoping (per-tenant country, currency, locale).

- Multi-tenant SaaS white-label branding.

2.3 Channels

- **Web** channel (consumer web) and **app** channel (mobile-responsive PWA installable).
- **Agent shop portal** for shop owners to manage their shop & publish products.
- **Admin portal** for overall management + monitoring.

2.4 Deployment & operations (production SaaS)

- Deployable to **AWS, Kubernetes, on-prem, Docker** — horizontal **autoscaling**, easy deploy.
- Buy-as-software, self-host, or **full managed service with hosting**.
- Mostly **free + standard** production SaaS-level tooling.

2.5 Testing / API tooling

- Use **WireMock** for offline provider/payment stubbing where anything cannot connect.
- Provide **test** API documentation and **production** API documentation.

2.6 Support & 24/7 operations

- **Support service** and a **24/7 support portal with management**.
- Include all **e-services** needed for a production application.
- Development **release notes**, **QA release notes**, and every document needed start-to-end: architecture diagrams, manuals, **SDLC process**, as a standard company.

2.7 Automation / proactive capabilities

- **Auto-detect issues** (live + dev + QA).
- **Fraud detection system**.
- **Auto-fix** live, dev and QA issues where feasible.
- **Full automation with a support system**.
- Monitoring, health checks, observability.

3. Product-name & business direction guidance

- Blueprint product name: **TicketMesh** — "All your tickets, one platform."
- Multi-product company portfolio later (e-commerce, hotel, e-channel, fintech, daycare, learning, care, portals, apps) — all reuse the same **platform prompt + architecture + docs**. Ticketing is built first; each product mirrors it.
- Sell through multiple channels like the WordPress marketplace model.

3.5 Delivery decision & blueprint coverage

- **Delivery decision:** the blueprint's multi-microservice topology is delivered this build as a **pragmatic modular single-backend** (ticketmesh-core Spring Boot) with clear within-core module boundaries (identity, catalog, provider, search, inventory, reservation, booking, payment, ticket, notification, fraud, analytics, AI/ML). This stays coherent and shippable while remaining microservice-ready.
- **Coverage mapping:** see docs/blueprint-coverage.md for the honest per-part status.

4. Non-negotiable (from CLAUDE.md — authorship/quality)

- Author: ****sangiya**** (asan935para@gmail.com). No "Claude"/"AI-generated" anywhere.
- Production-grade only; real tests with real assertions; professional repo names.
- Data safety: never commit real production data, credentials, or customer info.
- No employer proprietary content.

Author

Prepared by sangiya — Associate Software Architect.